FLINK FORWARD #ASIA 2021 ONLINE 实时即未来 REAL-TIME IS THE FUTURE

美团基于流批一体构建增量数仓生产实践

The Production Practice of Building Incremental Data Warehouse based on Unified Architecture at Meituan

刘迪珊

FLINK FORWARD #ASIA 2021 ONLINE 实时即未来 REAL-TIME IS THE FUTURE

#1 项目目标

#2 核心设计与实现

#3 计算能力优化

#4 未来展望

FLINK FORWARD #ASIA 2021 ONLINE 实时即未来 REAL-TIME IS THE FUTURE

#1 项目目标

数仓生产面临的问题

#1

海量数据就绪时间难保证：对于天级海量数据，离线生产性能优化的空间近乎达到瓶颈

#2

实时生产准确性依赖状态容量：状态越大、快照制作和恢复的成本越高，时效性和准确性间权衡

#3

离线实时两套计算逻辑，口径难统一：离线实时数据生产，采用两套技术栈，产出两套数据

数仓增量生产目标

离线数仓 → 增量数仓

本期目标：

1）数仓生产时效性提升：T+1->分钟级，特征生产主流程落地

2）建立同时支持批量和增量读写的存储引擎

3）Flink支持对上述存储引擎的增量读写

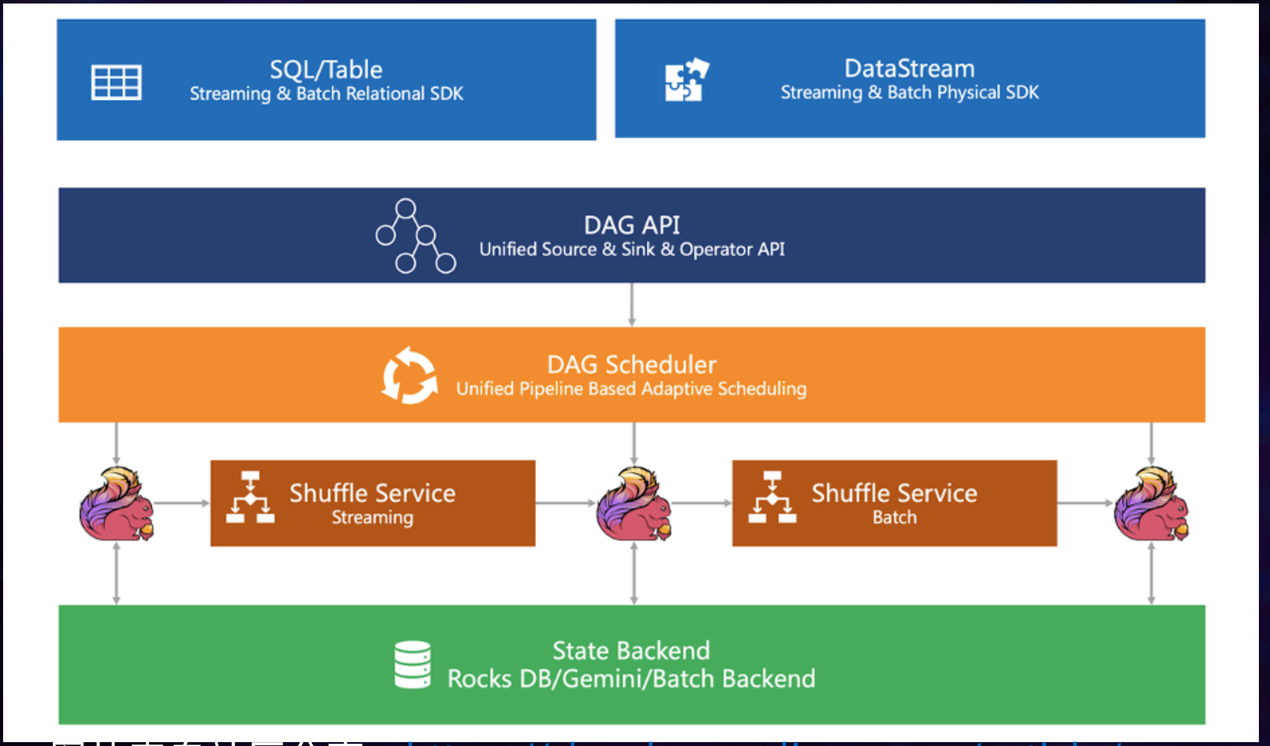
4）提升Flink在百万QPS/10TB状态下，流计算任务的处理能力和稳定性

FLINK FORWARD #ASIA 2021 ONLINE 实时即未来 REAL-TIME IS THE FUTURE

#2 核心设计与实现

需求背景

1 Flink作为增量生产的计算引擎

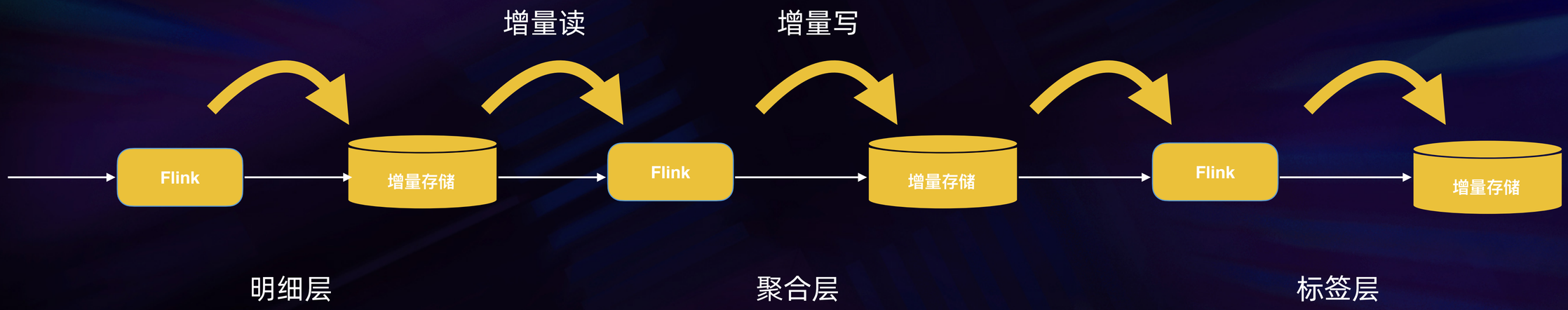


图片来自社区分享：https://developer.aliyun.com/article/783112

- Flink流批统一架构，一套代码、统一计算口径

需求背景

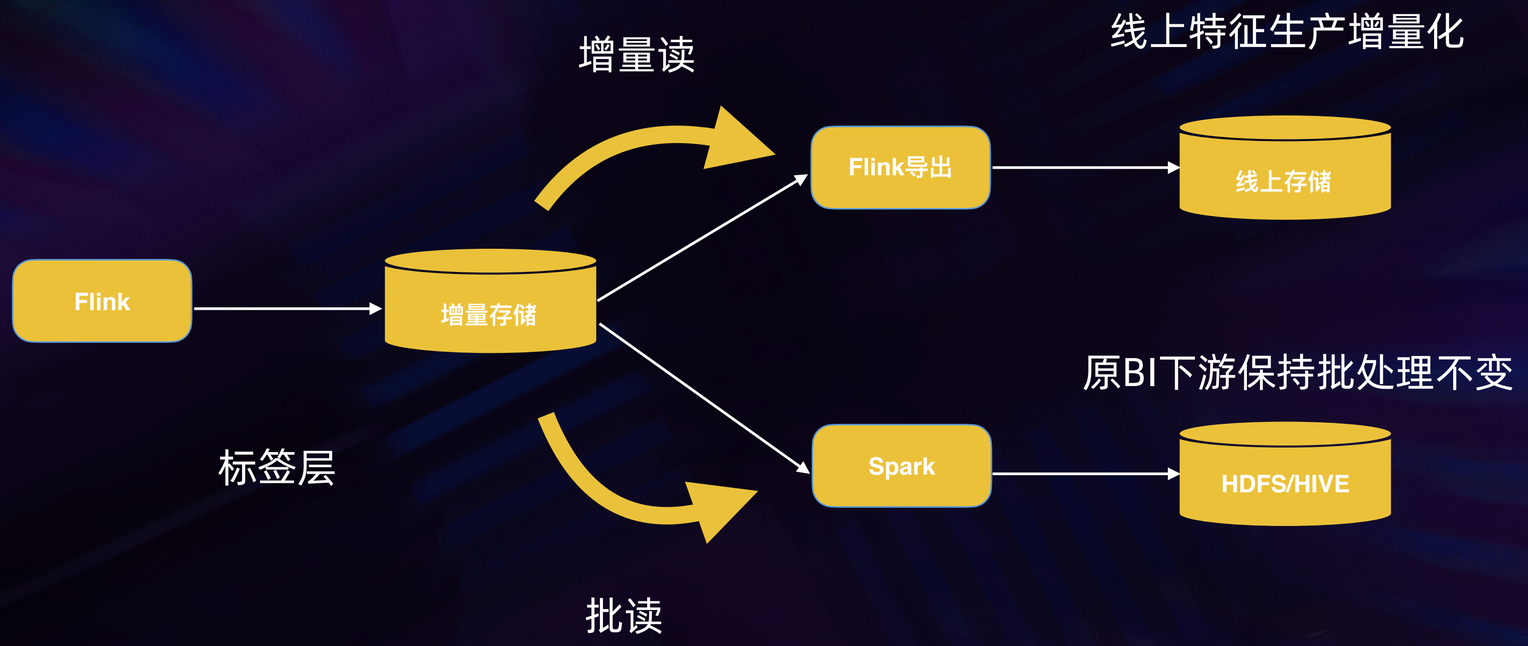
2 增量读写的必要性



- 离线数仓分层划分，目标增量化表的上游链路上的所有节点也必须增量化

需求背景

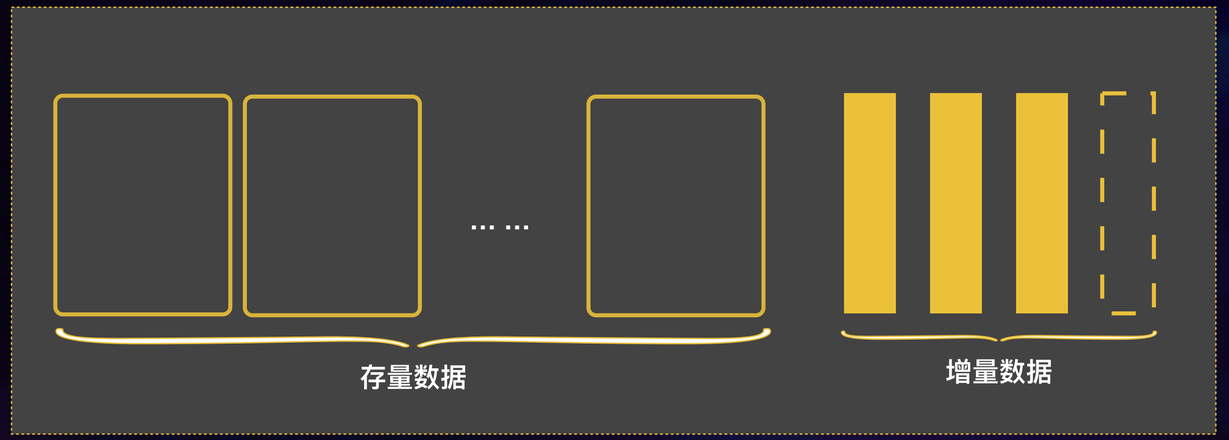
3 批读的必要性



- 终极目标：增量生产架构替换原有离线生产架构
- 增量生产架构：不是所有节点都一定是增量化的，而是这套架构能同时支持增量和批计算

需求背景

4 批写的必要性



- 存量数据，批写初始化
- 数据修复，批量更新

需求背景

5 Upsert的必要性

存量数据

id	value
1	100
2	200
3	300
4	400
…	…
100	1000

+

增量数据

id	value
1	120
2	210
101	10100

最新数据

id	value
1	120
2	210
3	300
4	400
…	…
101	10100

- 增量数据需要与全量数据进行merge
- 增量数据远少于全量数据，若不支持Upsert，读取全量数据效率低

需求背景

6 before/after的必要性

CDC变更数据（binlog）输出：
更新外存 id, pay_status, delivery_status, poi

UPDATE, 100, 1, 0, 1

UPDATE, 101, 1, 0, 2

UPDATE, 101, 1, 1, 2

orders表
（主键：id）

id	pay_status	delivery_status	poi
100	1	0	1
101	1	1	2

changelog流

+ 100, 1, 0, 1

+ 101, 1, 0, 2

- 101, 1, 0, 2

+ 101, 1, 1, 2

```
SELECT COUNT(id) AS cnt
FROM orders
WHERE pay_status = 1 AND
delivery_status != 1
GROUP BY poi;
```

输出：更新外存
poi, cnt

UPDATE 1, 1

UPDATE 2, 1

UPDATE 2, 0

poi表
（主键：poi）

poi	cnt
1	1
2	0

- 增量生产的表既会作为结果查询，又会作为作为下游增量生产的输入
- 对于一个增量任务，为了保证结果数据的准确性，需要依赖before/after进行回撤

需求背景

7 事务的必要性（Exactly once）

- 存储与计算联动回滚，否则会出现重复计算
- 存储层要支持事务提交，并与计算checkpoint保持一致

设计假设与权衡

FLINK FORWARD #ASIA 2021 ONLINE 实时即未来 REAL-TIME IS THE FUTURE

1 表存储均有主键

自增主键

id	value
1	100
2	200
3	300
4	400
…	…
100	1000

复合主键

session	seqNo	value
1	1	100
1	2	400
2	2	200
2	3	300
…	…	500
100	1	100

设计假设与权衡

2 存储具备产生before/after的能力

orders表
（主键：id）

id	pay_status	delivery_status	poi
100	1	0	1
101	1	1	2

changelog流

+ 100, 1, 0, 1
+ 101, 1, 0, 2
- 101, 1, 0, 2
+ 101, 1, 1, 2

```
SELECT o.id, o.pay_status,
o.delivery_status , p.type
FROM orders AS o
JOIN poi FOR SYSTEM_TIME AS OF
o.proc_time AS p  ON o.poi = p.id;
```

orders_detail表
（主键：id）

id	pay_status	delivery_status	type
100	1	0	10
101	1	1	10

changelog流

+ 100, 1, 0, 10
+ 101, 1, 0, 10
- 101, 1, 0, 10
+ 101, 1, 1, 10

```
SELECT COUNT(id) AS cnt
FROM orders_detail
WHERE pay_status = 1 AND
delivery_status != 1
GROUP BY type;
```

- 计算模块产生变更数据需要依赖Flink状态，会增加计算/运维成本，增加故障恢复时间
- 若只由计算产生变更数据，随着计算深度的增加，数据会呈指数放大。下游计算压力变大，资源消耗增加

设计假设与权衡

3 单表由一个任务写入

orders_detail表
（主键：id）

```
SELECT o.id, o.pay_status,
o.delivery_status , p.type
 FROM orders AS o
JOIN poi FOR SYSTEM_TIME AS OF
o.proc_time AS p  ON o.poi = p.id;
```

id	pay_status	delivery_status	type
100	1	0	10
101	1	1	10

changelog流

+100，1，0, 10

+101，1，0, 10

-101，1，0, 10

+101，1，1, 10

- 多个任务同时更新同key数据，会出现数据一致性问题。除非存储层进行排序
- 一个任务出现故障回滚时，还需要其他任务进行事务的协调

设计假设与权衡

4 计算任务保证消息按主键的有序性

- 不同实例写同key数据，同样会有顺序性问题，产生的before/after可能会不一致，造成计算结果不一致
- 顺序性要求与计算场景更为紧密，排序前置灵活性更好

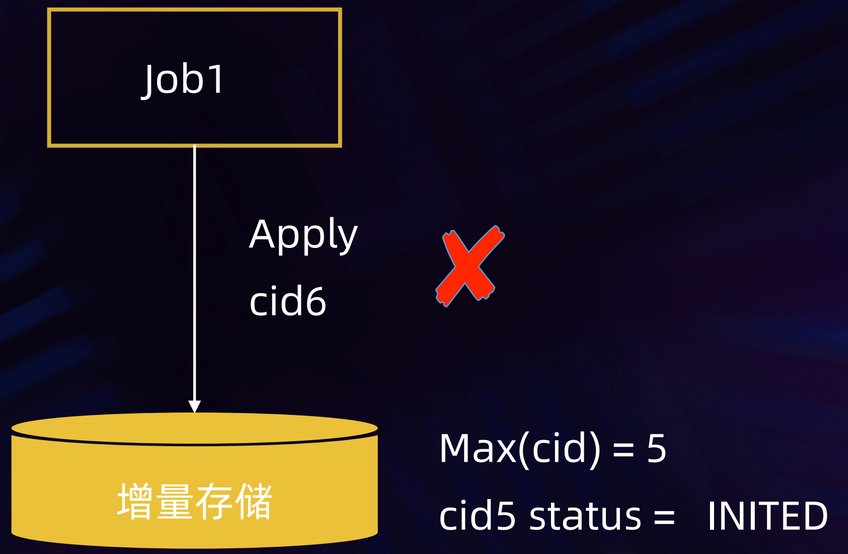
设计假设与权衡

5 存储负责产生和提供事务ID

- 可扩展性，考虑未来支持多任务更新单表不同列子集
- 存储产生递增事务ID更直接和统一，能避免跨任务间的事务协调

设计假设与权衡

6 计算顺序提交，并负责对失败的事务进行回滚



- 事务ID是递增的
- 客户端顺序申请和提交，并对故障的事务进行回滚
- 当前最大事务ID对应状态不是终态时（已提交或已回滚），不允许客户端申请新的事务ID

增量存储关键能力与设计权衡总结

FLINK FORWARD #ASIA 2021 ONLINE 实时即未来 REAL-TIME IS THE FUTURE

写能力

批写（初始化）

增量插入/更新/删除

读能力

对齐批读接口

增量读变更（before/after）

事务性

具备分钟级事务提交能力

对于未成功提交的事务，具备回滚能力

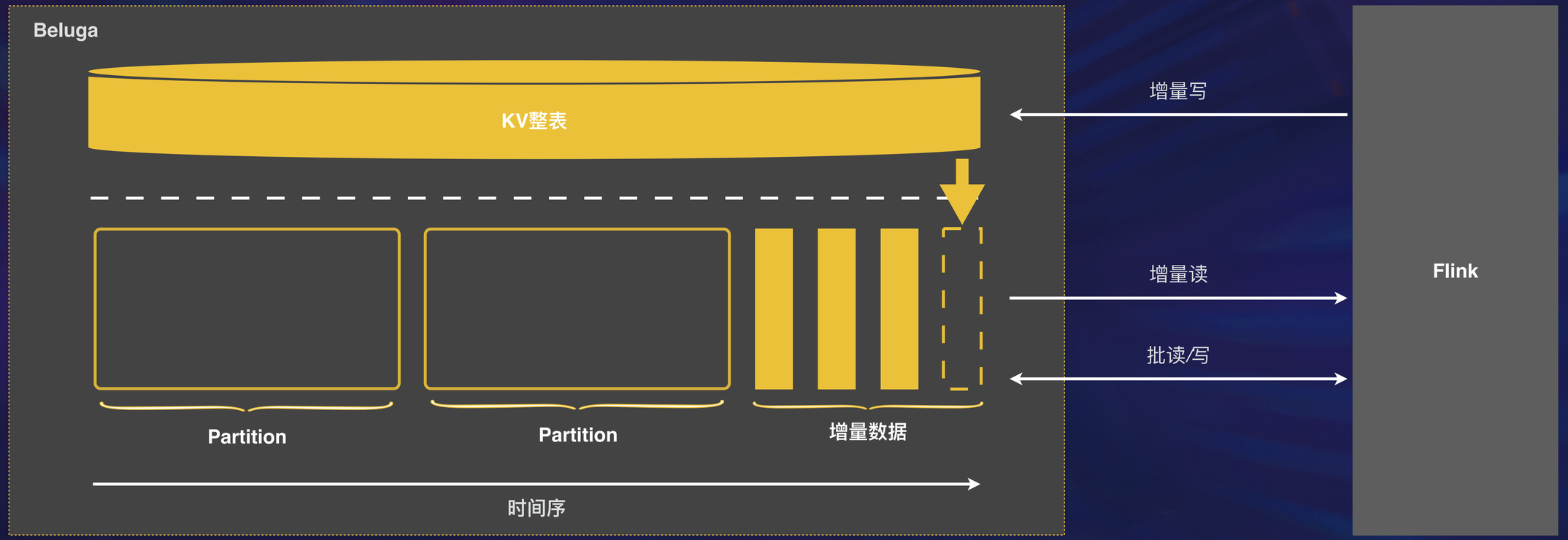
设计假设与权衡：

1. 所有表均有主键
2. 存储具备产生before/after的能力
3. 单表由一个任务写入
4. 计算保证消息按主键的有序性
5. 存储负责产生和提供事务ID
6. 计算顺序提交，并负责对失败的事务进行回滚

数据湖存储

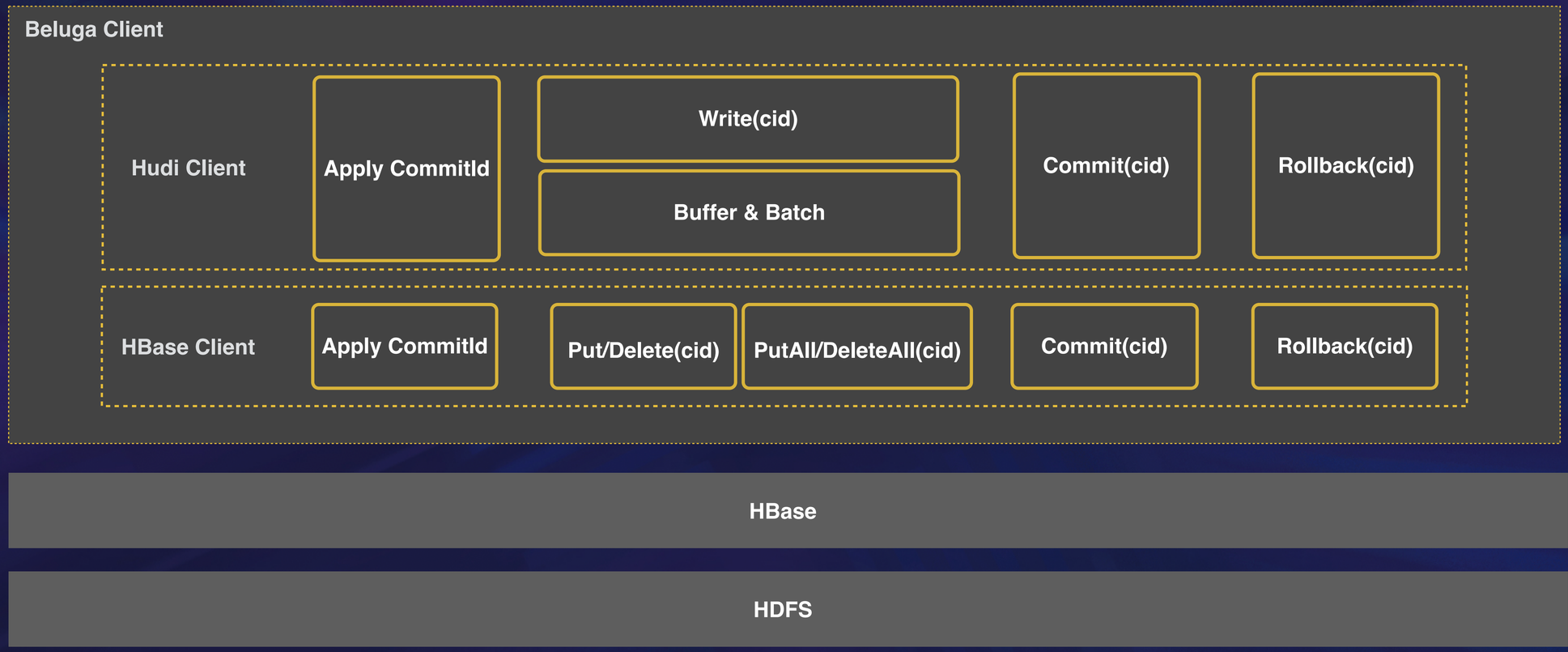
- 普遍支持批量和增量读写
- 普遍支持多版本与事务性
- 多种底层存储，数据表逻辑组织结构

增量存储Beluga整体架构



- 客户端API：面向应用程序，提供读写接口、事务接口
- KV层：基于HBase改造。支持按主键插入、更新和删除数据；负责生成before/after数据
- Storage层：基于Hudi改造。集成成熟读写接口和设计，支持增量读写和批量读写

Beluga 客户端API



- 客户端对外接口与Hudi类似，保留嵌入式设计
- 增量写时，为生成全表快照（获取before/after），由原先直接写HDFS，改为先调用HBase接口
- 增量和批读取，复用Hudi能力

Beluga 增量写-KV层数据

1 HBase提交状态表（Commit Status Table），维护事务当前的状态，用于事务提交与回滚

- 表region级别的commit信息（第一阶段）
- 表级别的commit信息（第二阶段），表当前最大的commitid

提交状态表（Commit Status Table）

RowKey	Column Family detail	Column Family cf
tablename+cid1		cf:status=“committed”
tablename		cf:max=“cid1”
tablename+cid1+regionid2	detail:status=“committed”	
tablename+cid1+regionid1	detail:status=“committed”	
tablename+cid2		cf:status=“inited”

Beluga 增量写-KV层数据

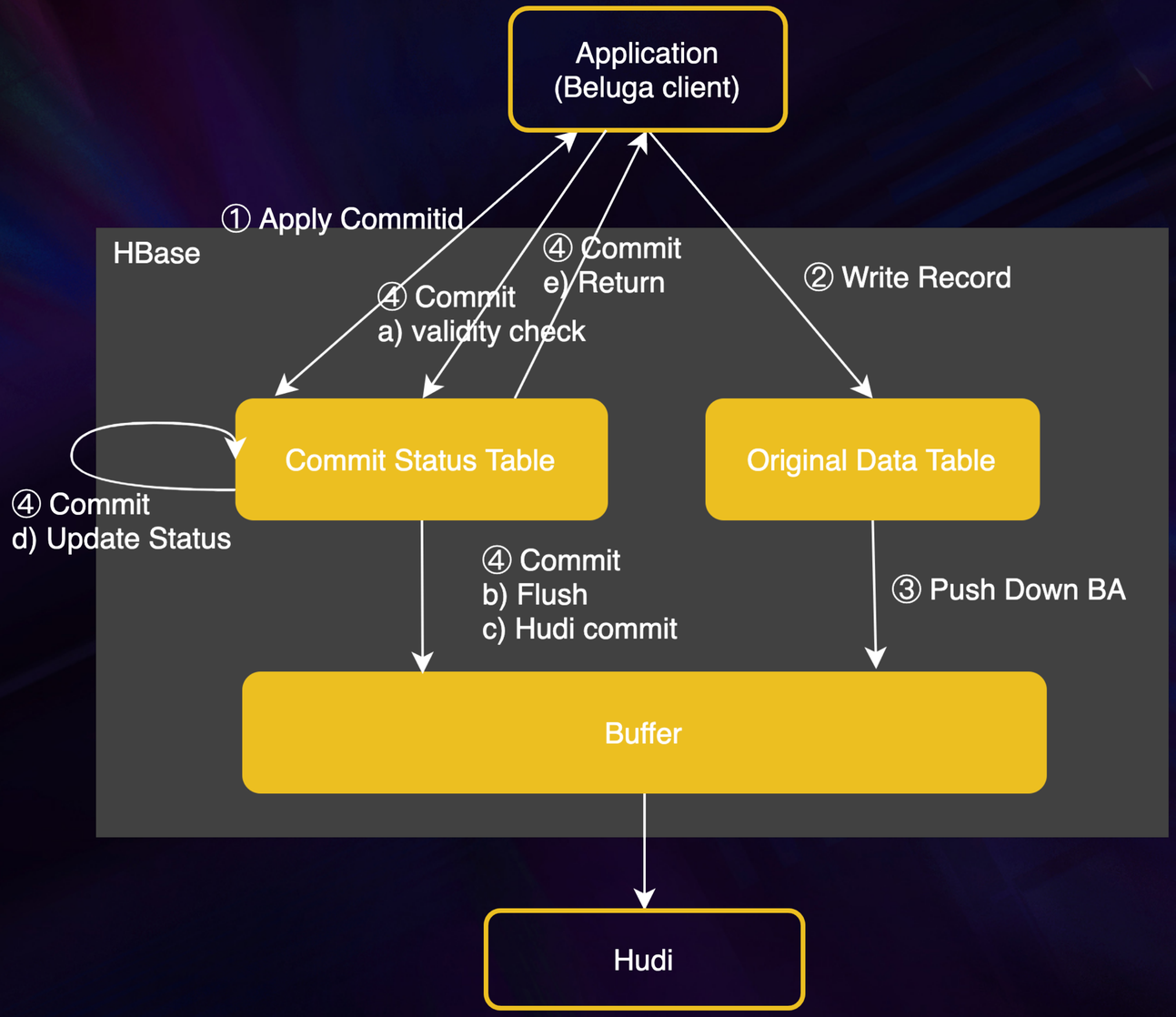
2 HBase原始数据全量表（Original Data Table），支持同主键在一个事务中更新多次

- commitid保存到元数据tag中，支持一个事务中同key多次写入
- 多版本保留时间为天级（7天），按写入时间超过版本保留时间，进行压缩
- 使用TimeStamp+SeqNo，保证同一个事务内同key多次写入的顺序

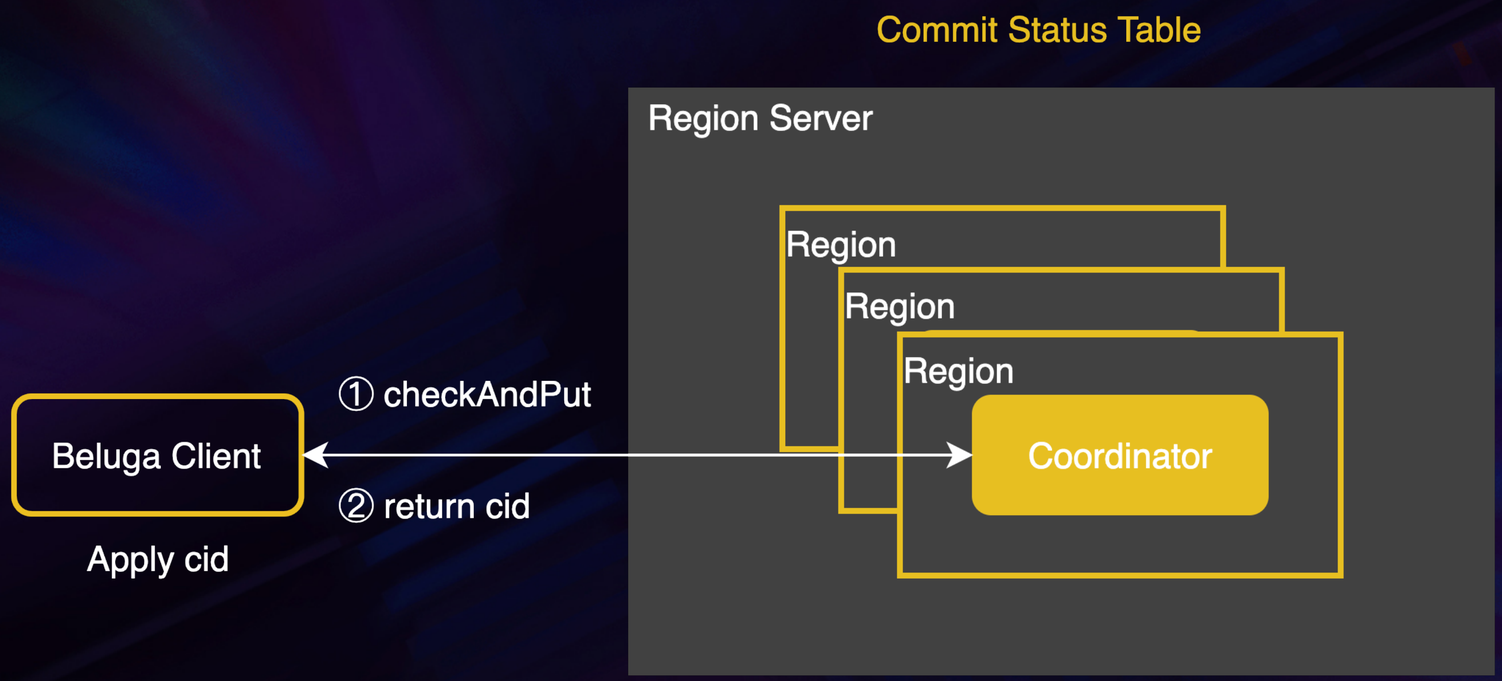
原始数据全量表（Original Data Table）

RowKey	Tag	TmeStamp	Column Family cf
key1	cid1	t4	cf:value="… …"
key2	cid1	t3	cf:value="… …"
key1	cid1	t2	cf:value="… …"
key1	cid2	t1	cf:value="… …"

Beluga 增量写-整体流程

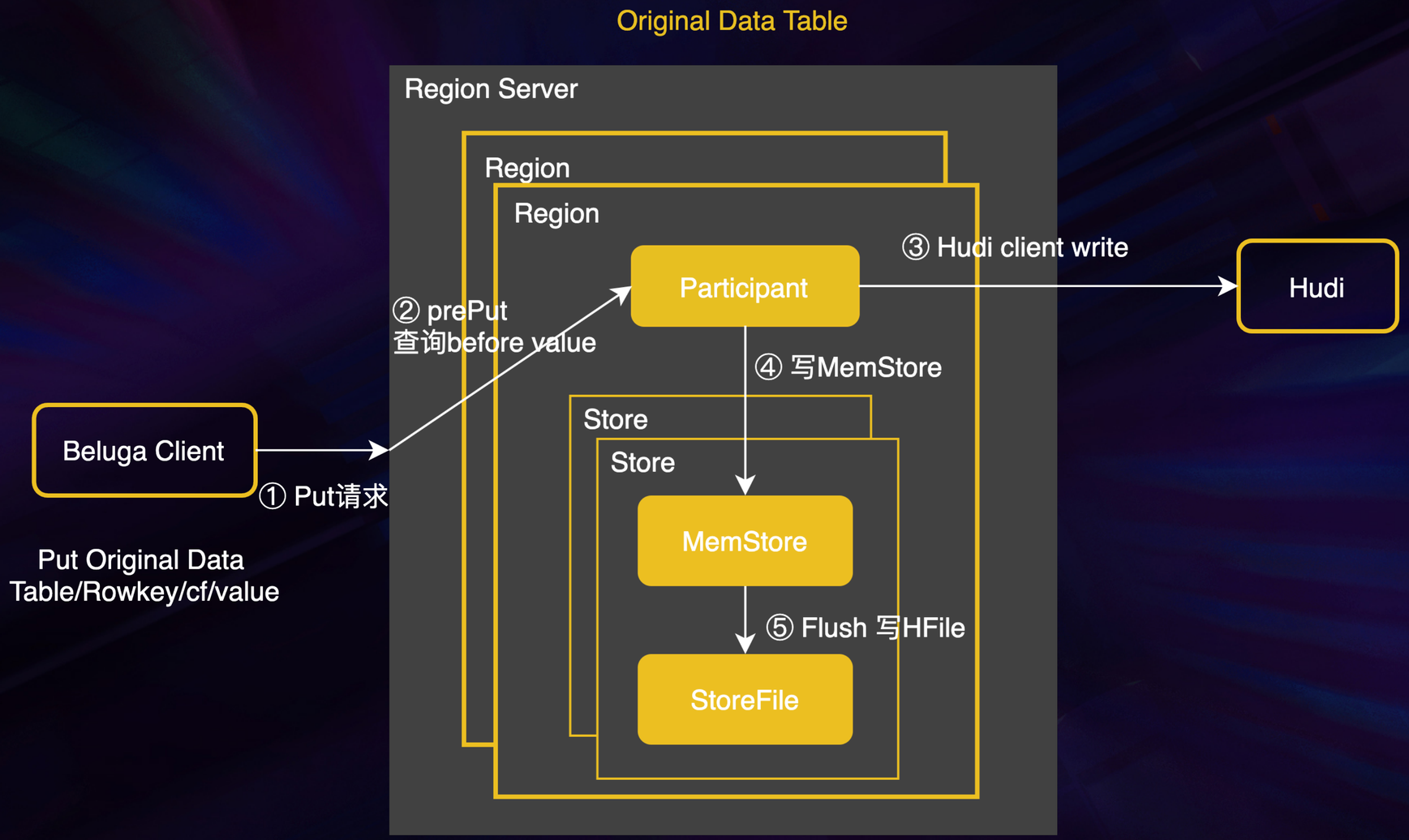


Beluga 增量写-事务开启



- tablename，max commitid++
- tablename+cid，status=INITED

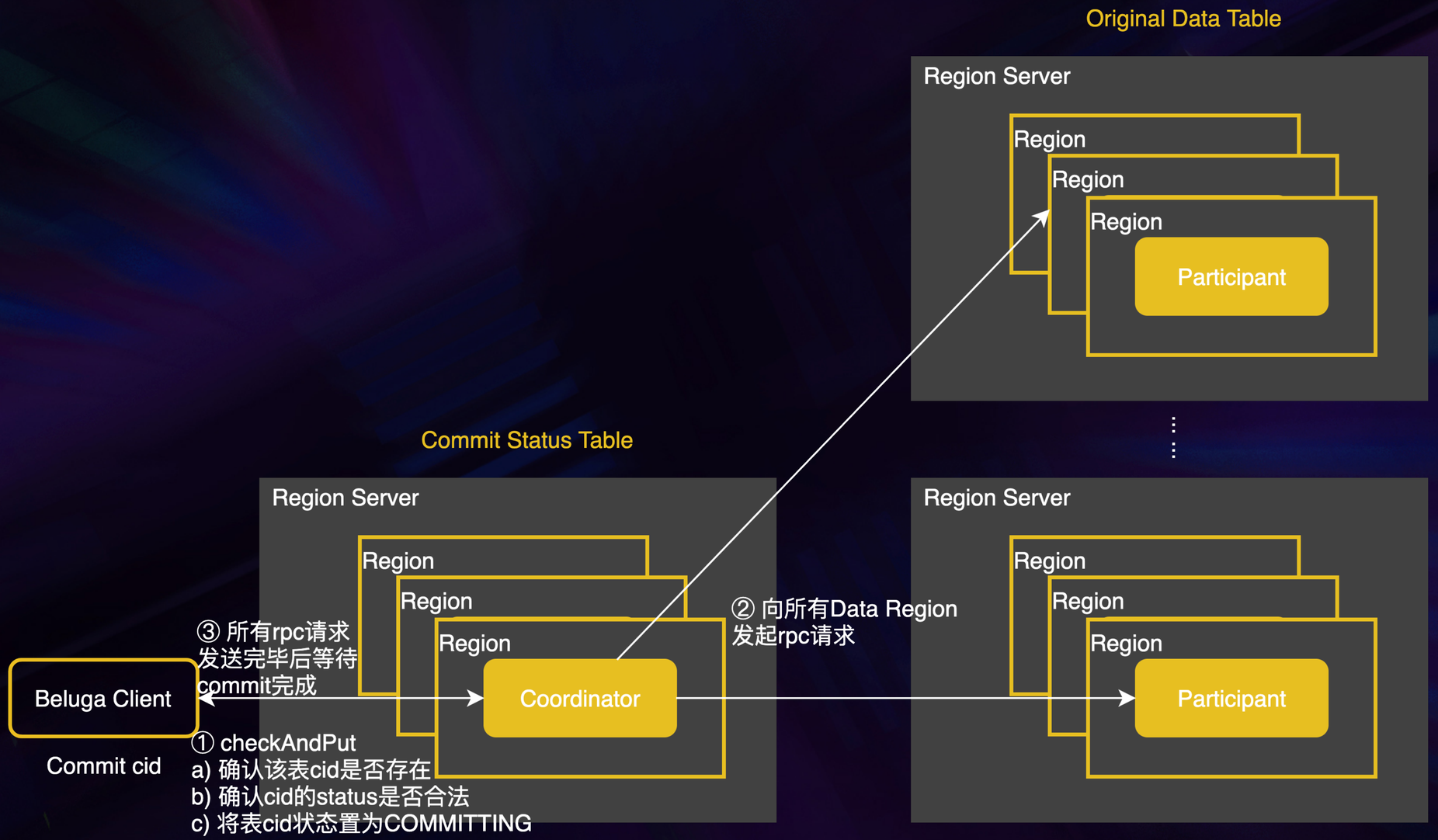
Beluga 增量写-数据写入



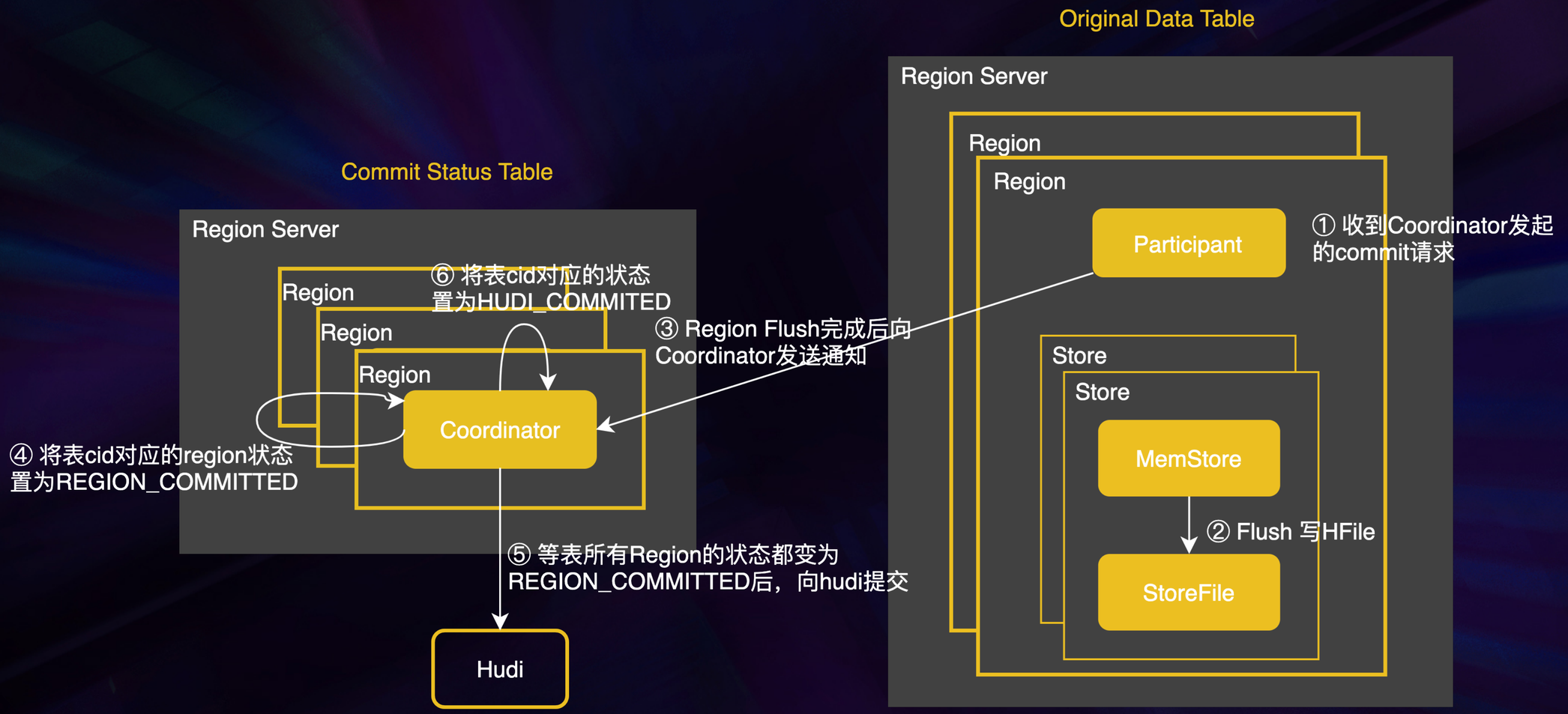
Beluga 增量写-文件合并

- 自定义合并策略，优先对commit内文件数量超过阈值的文件进行合并
- 越旧的commit，合并跨度越大；越新的commit保持独立性，方便快速回滚

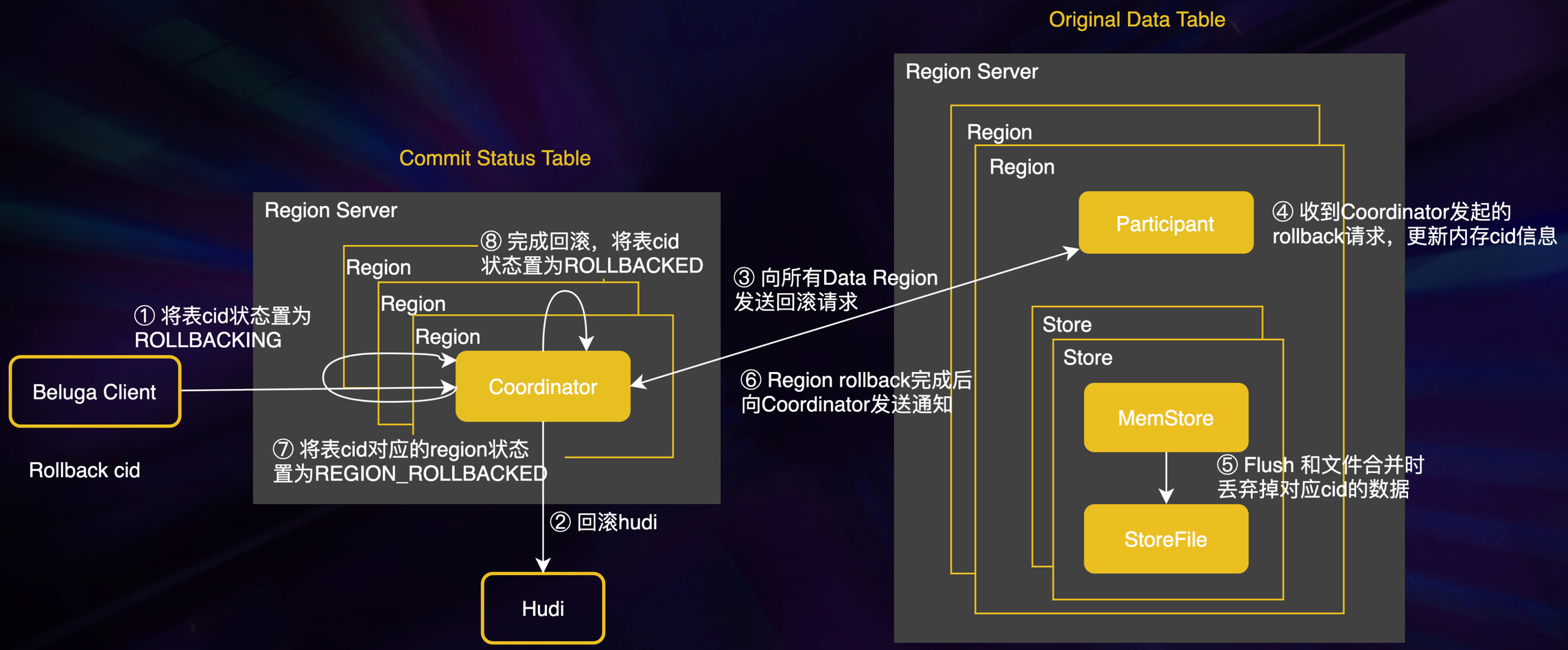
Beluga 增量写-事务提交



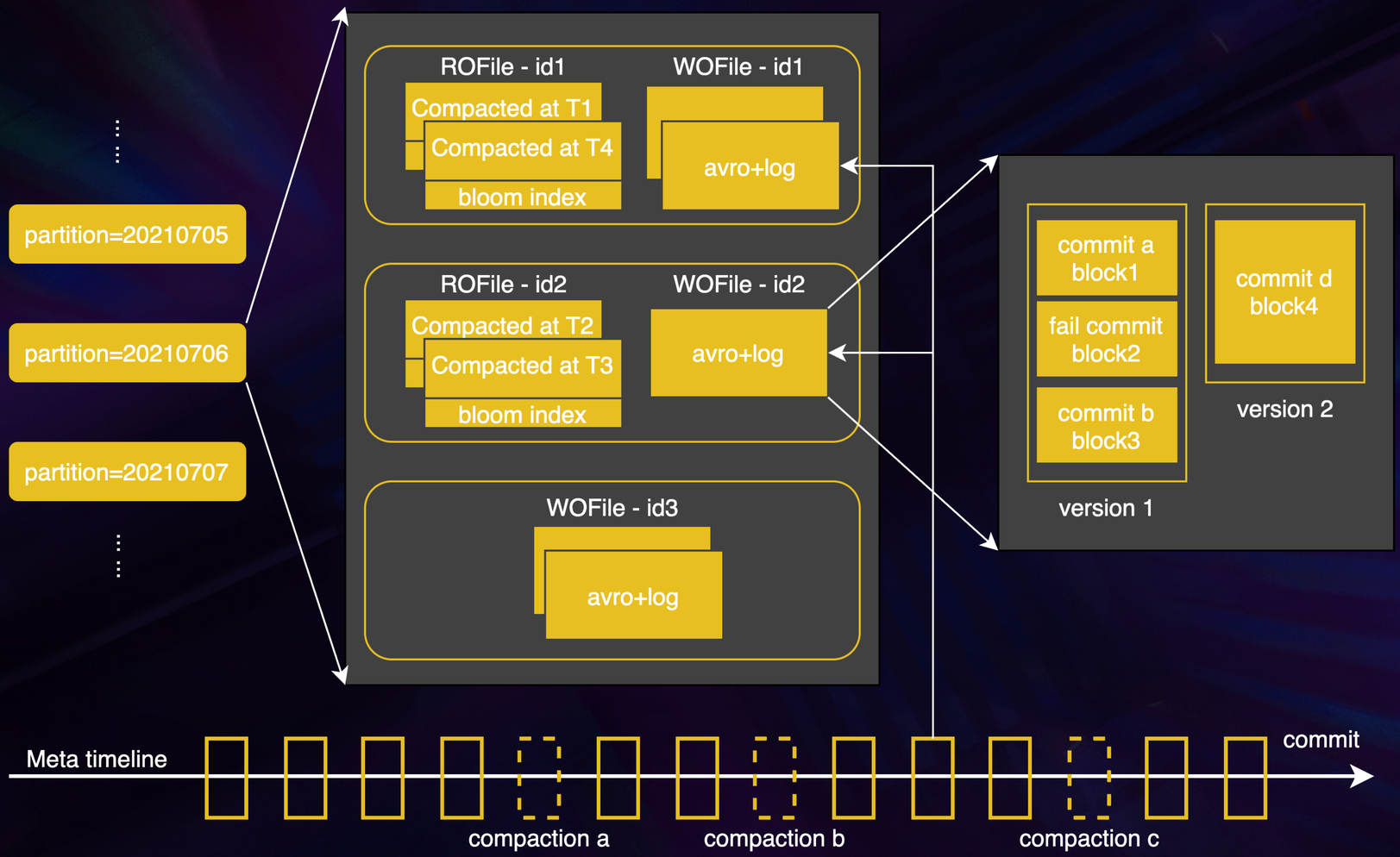
Beluga 增量写-事务提交



Beluga 增量写-事务回滚



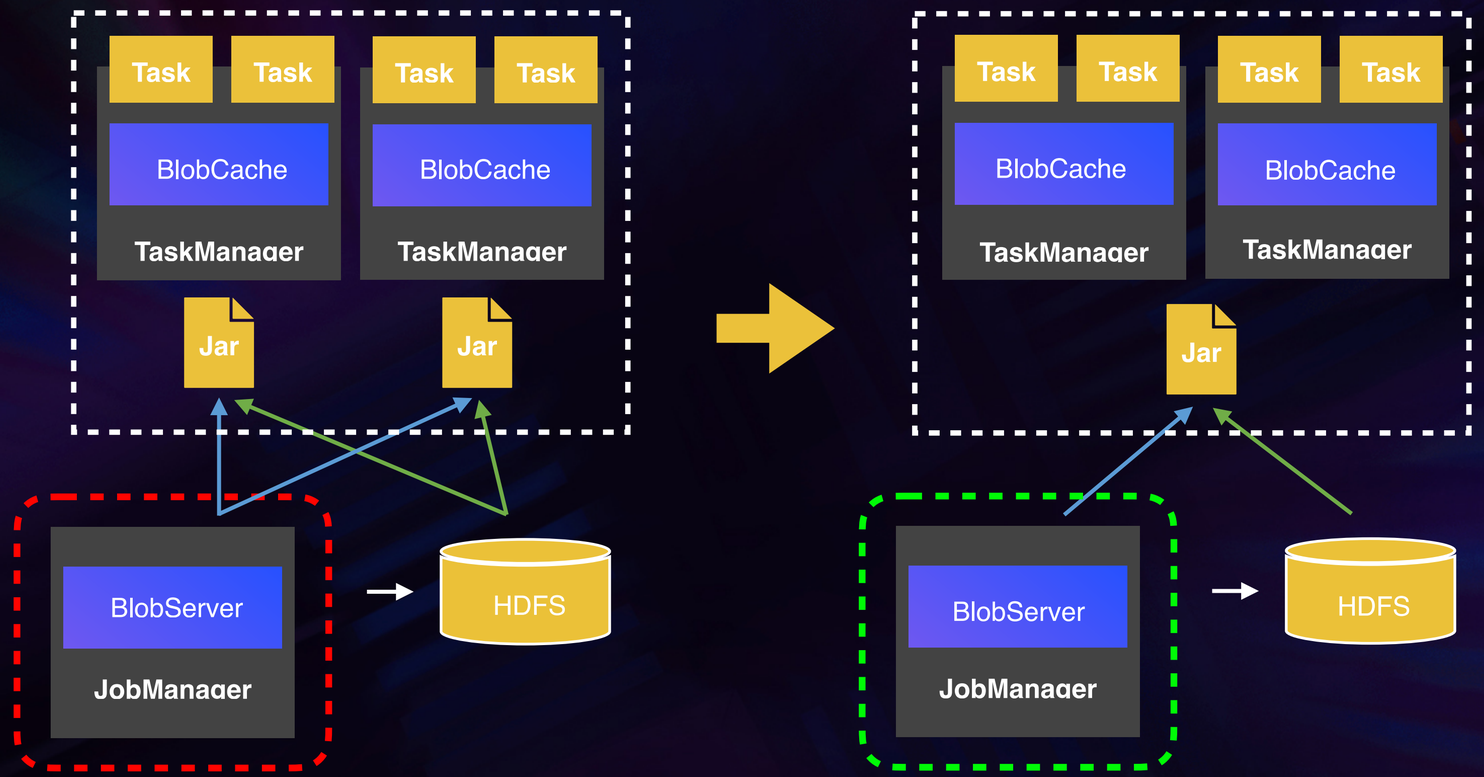
Beluga Storage层持久化



FLINK FORWARD #ASIA 2021 ONLINE 实时即未来 REAL-TIME IS THE FUTURE

#3 计算能力优化

大作业与状态稳定性优化

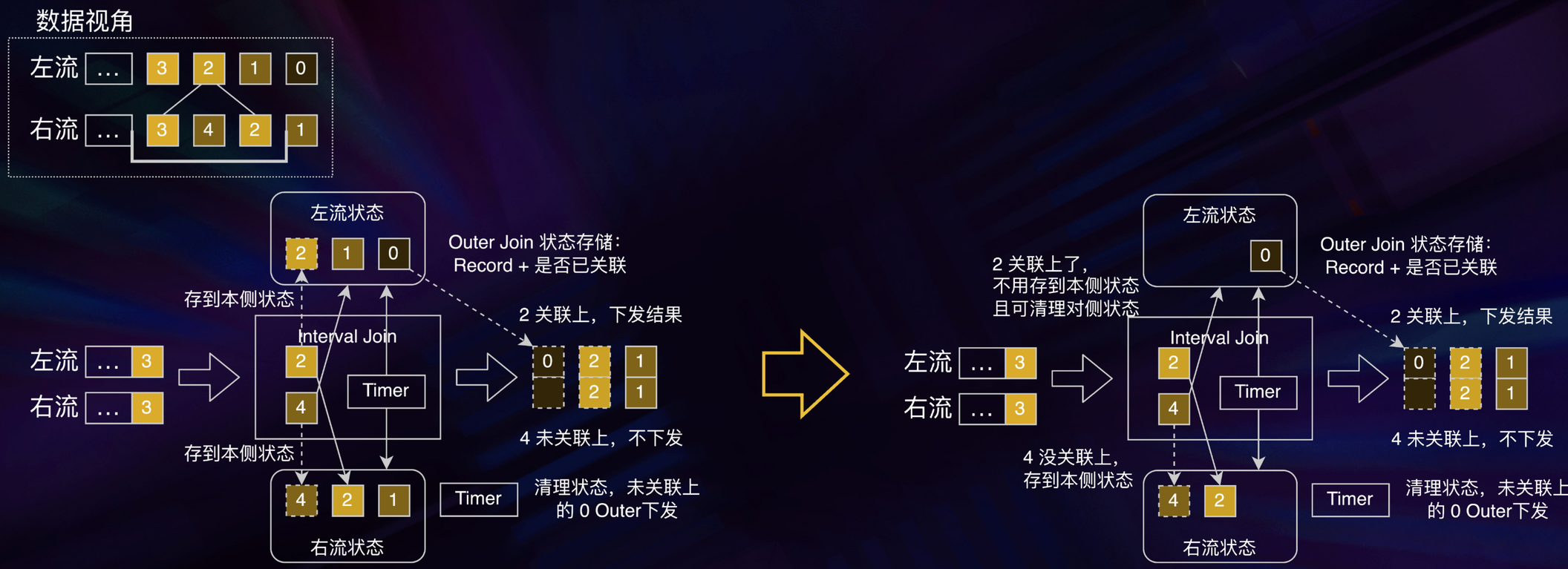


美团Flink大作业部署与状态稳定性优化实践

生产实践 01月08日 17:20 - 18:00

冯斐 王非凡

Join算子优化



Flink Join算子优化

核心技术 01月08日 17:20 - 18:00

孙梦瑶

FLINK FORWARD #ASIA 2021 ONLINE 实时即未来 REAL-TIME IS THE FUTURE

#4 未来展望

未来展望

1）Beluga优化：kv与storage层存储整合，减少存储冗余/IO和rpc交互

2）降低数据可见性延迟

3）Beluga支持点读

4）Flink计算状态容量扩展，利用增量存储进行冷数据的访问更新

FLINK FORWARD
#ASIA 2021
ONLINE
实时即未来
REAL-TIME IS THE FUTURE
THANKS